v1.0.0
20260309

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[Lab Notes]

THREAD COORDINATION IN JAVA - LIBRARY SUPPORT

CONCURRENT BUILDING BLOCKS

- The Java platform libraries (Java 5.0 & Java 6.0) include a rich set of concurrent building blocks such as thread-safe collections and a variety of synchronizers that can coordinate the control flow of cooperating threads
  - **Synchronized Collections**
  - **Concurrent Collections**
  - **Synchronizers**

SYNCHRONIZED COLLECTIONS

- *Synchronized wrappers*
  - created by `Collections.synchronizedXXX` factory methods
  - achieving thread-safety by
    - encapsulating the state
    - synchronizing every public method
  - > achieving safety by serializing all access to the collection's state
- Problems
  - need to use additional client-side locking to guard compound actions
    - common compound actions include iteration, navigation, conditional operations such as put-if-absent
  - the object to be used for client-side locking is the synchronized collection object itself
  - performance problems
    - locking the collection for long-term operations, such as iteration...
    - strongly limiting concurrency

CONCURRENT COLLECTIONS

- Introduced with Java 5.0 and *designed for concurrent access* from multiple threads
  - greatly improving scalability and performance with respect to synchronized collections
- Main classes
  - **`ConcurrentHashMap`**
    - replacement for synchronized hash-based `Map` implementations
  - **`CopyOnWriteArrayList`**
    - a replacement for synchronized `List` implementations
  - **`Queue`** and **`BlockingQueue`**
    - interfaces with a different kinds of implementations available

BLOCKING QUEUE

- Provides blocking **put** / **take** methods + timed equivalent **offer** / **poll**
  - if the queue is full, put blocks until space becomes available
  - it the queue is empty, take blocks until an element is available
- Queue can be **bounded** and unbounded
  - unbounded queue are never full
- Bounded queue as a basic building block for *producer-consumer design pattern*
  - powerful resource management tool for building reliable applications
    - making programs more robust to overload by throttling activities that threaten to produce more work than can be handled
- Different classes implementing `BlockingQueue`
  - **`LinkedBlockingQueue`**, **`ArrayBlockingQueue`**, **`PriorityBlockingQueue`**,...

DEQUES AND WORK STEALING

- **`Deque`** and **`BlockingDeque`** data structure
  - introduced with Java 6.0
  - double-ended queue that allows for efficient insertion and removal from both the head and the tail
  - implementations: `ArrayDeque` and `LinkedBlockingDeque`
- Used for *work stealing* design pattern
  - similar to producers-consumers
  - each consumer has its own deque
  - if a consumer exhausts the work in its own deque, it can steal work from the *tail* of someone else's deque
- More scalable than producers-consumers
  - workers don't contend for a shared work queue
    - most of the time they access only their own deque, reducing contention
  - when accessing to others' deque, the access is from the tail, not from the head
    - further reducing contention

AGGREGATE OPERATIONS AND LAMBDA

- The Collections Framework has undergone a major revision in Java 8 to add aggregate operations based on the newly added streams facility and lambda expressions.
- As a result, the ConcurrentHashMap class introduces over 30 new methods in this release
  - forEach methods
    - forEach, forEachKey, forEachValue, and forEachEntry
  - search methods
    - search, searchKeys, searchValues, and searchEntries
  - a large number of reduction methods
    - reduce, reduceToDouble, reduceToLong etc.

SYNCHRONIZERS

- A *synchronizer* is any object that coordinates the control flow of threads based on its state
  - blocking queue can function as synchronizers
- Very important building blocks of concurrent applications
  - passive component encapsulating coordination functionalities
- All synchronizers share certain structural properties
  - encapsulating state that determines whether threads arriving at the synchronizers should be allowed to pass or forced to wait
  - providing methods to manipulate that state
  - providing methods to wait efficiently for the synchronizer to enter in the desired state
- Main types provided with Java library
  - **Locks**
  - **Semaphores**
  - **Latches**
  - **Barriers**

LOCKS

- Providing explicit lock functionality
  - vs. intrinsic lock given by synchronized blocks
- **`Lock`** interface and **`ReentrantLock`** implementation

```
public interface Lock {
  void lock();
  void lockInterruptibly() throws InterruptedException;
  boolean tryLock();
  boolean tryLock(long timeout, TimeUnit unit) throws InterruptedException;
  void unlock();
  Condition newCondition();
}
```

- Typical usage:

```
Lock lock = new ReentrantLock();
...
lock.lock();
try {
  // update shared object state
  // catch exception and restore invariants if necessary
} finally {
  lock.unlock();
}
```

POLLED AND TIMED LOCK ACQUISITION

- Using **`tryLock`** for polled and timed lock acquisition to have more sophisticated error recovery

```
public boolean transferMoney(Account from, Account to, Amount am)
                    throws InsufficientFundException, InterruptedException {
  while (true) {
    if (from.lock.tryLock()){
      try {
        if (to.lock.tryLock()){
          try {
            if (from.getBalance().compareTo(am)<0) {
              throw new InsufficientFundException();
            } else {
              from.debit(am);
              to.credit(am);
              return true;
            }
          } finally {
            to.lock.unlock();
          }
        }
      } finally {
        from.lock.unlock();
      }
    }
  }
}
```

EXPLICIT VS. INTRINSIC LOCKS

- Intrinsic locking (synchronized blocks) works fine in most situations but has some functional limitations
  - it is not possible to interrupt a thread waiting to acquire a lock..
  - ..or to attempt to acquire a lock without being willing to wait it forever
- In this case explicit locks can be used...
  - managing interruption
  - specifying bounded wait time
- ..with a strong discipline that must be followed by the programmers
  - explicit unlocking locks, for every possible scenario
- Performance comparison
  - in Java 5.0 explicit locks outperform intrinsic locks
    - `ReentrantLock` throughput about 4 times than intrinsic lock
  - in Java 6.0 same performance

SEMAPHORES

- Implementation of Dijkstra's basic semaphore construct
- **`Semaphore`** class
  - created specifying a number of virtual *permits*
  - **`acquire`** + **`release`** method
  - possibility to enforce *fairness*

LATCHES

- A *latch* is a synchronizer that can delay the progress of a thread until it reaches its *terminal* state
- Function as a *gate*
  - until the latch reaches the terminal state, the gate is closed and no thread can pass
  - in the terminal state the gate opens allowing all threads to pass
  - once the latch reaches the terminal state, it cannot change the state again and so it remains open forever
- **`CountDownLatch`** class
  - `CountDownLatch(int count)`
    - to initialize the latch with a specific count
  - **`countDown`**
    - method to decrement the count
  - **`await`**
    - method that causes the current thread to wait until the latch has counted down to zero, unless the thread is interrupted.

LATCHES USE

- Used to ensure that certain activities do not proceed until other one-time activities complete.
- Main examples:
  - ensuring that a computation does not proceed until resources it needs have been initialized
    - using a binary latch for each resource
  - ensuring that a service does not start until other services on which it depends have started
    - using a binary latch for each service
    - starting service S would involve first waiting on latches for other services on which S depends, and then releasing the S latch after startup completes
  - waiting all parties involved in an activity (e.g: players in a multi-player game) are ready to proceed
    - the latch reaches its terminal state after all the players are ready

BARRIERS

A barrier is a synchronization method that forces a set of threads to stop at a specific point in execution and wait until all threads in the group have arrived. Only once the last thread reaches the barrier, they are all "released" to continue to the next phase.

- Implementation of the *barrier synchronization*
  - similar to latches in that they block a group of threads until some event has occurred
  - the key difference is that in this case all the threads must come together at a barrier point *at the same time* in order to proceed
  - > Latches are for waiting for *events*, barriers for other *threads*

- Latches (e.g., CountDownLatch): Primarily used to wait for events. A set of threads waits for a specific condition to be met. Once the latch is open, it cannot be reset.
- Barriers (e.g., CyclicBarrier): Primarily used to wait for other threads. It is used when multiple threads must perform a series of steps in lock-step. Unlike latches, barriers are cyclic: they can be reused after the threads are released.

- Implementation in Java: **CyclicBarrier**
  - constructors
    - CyclicBarrier(int numParticipants)
    - CyclicBarrier(int numParticipants, Runnable barrierAction)
  - await method